BOSS-KAMPF 2 · Welt 2 · Bearbeitungszeit 60 min · Hilfsmittel: Werkzeugkarten

W2·Boss – Das Turnier der Bogenschützen

Erreichbar: 24 BE · Papier-Compiler zu d) · Zusätzlich im Lab: **Online-Boss „Das Wettfischen am Drachensee“**
Niveau (KC 2017): Grund- und Leistungskurs – c) und die Analyse in e) (Effizienz) nur Leistungskurs (eA)

Neben dem Dungeon wartet eine Nebenquest: das große Turnier der Bogenschützen. Jede Schützin und jeder Schütze schießt sechs Pfeile, die erzielten Ringe werden addiert – wer die meisten Ringe hat, gewinnt. Euer Studio verwaltet das Turnier mit den Klassen **Schuetze** und **Turnier**. Die Klassen kennst du noch nicht – zeig, dass du Suchen, Sortieren und Zählen aus der Rangliste übertragen kannst!

Hinweise zur Bearbeitung

- Bearbeite alle Teilaufgaben **ohne Gerät** auf dem Antwortbogen am Ende dieses Hefts. Erlaubt sind nur deine Werkzeugkarten.
- Implementierungen schreibst du in **Java**, Struktogramme nach den Abi-Konventionen (Signatur, ←, „gib … zurück“).
- Zeitbudget wie im Abitur: ca. 2,5 Minuten pro BE. Die BE stehen rechts neben jeder Teilaufgabe.

Papier-Compiler

Deine Lösung zu **Aufgabe d)** tippst du direkt nach dem Boss-Kampf **unverändert** in der Online-IDE ab und startest die Tests im Testrunner. Alle Tests grün beim ersten Versuch: **Papier-Compiler bestanden**. Prüfe in d) und f) besonders die Leer-, Index- und null-Abfragen – das sind die typischen Abi-Fallen.

Zusätzlich im Lab: Online-Boss „Das Wettfischen am Drachensee“

Im **Algo Quest Lab** (algoquest.frankjuchim.de) wartet der Online-Boss von Welt 2: 25 Minuten Zeitlimit, Auswertung erst am Ende, der beste Versuch zählt. Die Punkte vergibt das Lab automatisch.

Aufgaben

a) Die Operation **sortieren** der Klasse Turnier ist in Abbildung 2 als Struktogramm dargestellt. Vor dem Aufruf enthält das Turnier die Schützen aus Abbildung 3.

Dokumentieren Sie in der Tabelle im Antwortbogen die Belegung der Reihung teilnehmer nach jedem Durchlauf der äußeren Schleife. Notieren Sie jeden Schützen mit Namen und Ringzahl. **[4 BE]**

b) Geben Sie an, um welches Sortierverfahren es sich bei der Operation sortieren handelt, und erläutern Sie seine Funktionsweise anhand des Struktogramms.

Begründen Sie, dass Ben nach dem Sortieren vor Lea steht, obwohl beide 52 Ringe erzielt haben. **[3 BE]**

c) Als Ringzahl-Vergleich zählt jede Auswertung des Ausdrucks `teilnehmer[j−1].getRinge() < teilnehmer[j].getRinge()`. Die Bedingung der solange-Schleife wird von links nach rechts ausgewertet; ist j > 0 falsch, wird der Ringzahl-Vergleich nicht mehr ausgeführt.

Geben Sie die Anzahl der Ringzahl-Vergleiche beim Sortieren der Belegung aus Abbildung 3 an. Ermitteln Sie die Anzahl der Ringzahl-Vergleiche im ungünstigsten Fall für 6 Schützen und für n Schützen und geben Sie für 6 Schützen eine Belegung der Ringzahlen an, bei der dieser Fall eintritt.

Die Turnierleitung stellt fest: „Das Sortieren von 1000 Schützen hat 0,5 Sekunden gedauert. Bei 2000 Schützen dauert es also etwa 1 Sekunde." Beurteilen Sie diese Aussage für den ungünstigsten Fall und gehen Sie dabei auch auf den Speicherbedarf der Operation sortieren ein.

[4 BE]

d) Nach dem Sortieren gehen noch Ergebnisse ein. Die Operation **nachtragen(s: Schuetze): Wahrheitswert** der Klasse Turnier fügt den Schützen s so in die absteigend nach Ringzahl sortierte Reihung teilnehmer ein, dass sie sortiert bleibt. Schützen mit geringerer Ringzahl rücken dazu jeweils um eine Position nach hinten; haben Schützen dieselbe Ringzahl wie s, wird s hinter ihnen eingefügt. Ist s null oder sind alle Plätze belegt, bleibt das Turnier unverändert und es wird falsch geliefert, sonst wahr.

Beispiel: Wird nach dem Sortieren aus a) der Schütze Jonas (Eibenhain, 52 Ringe) nachgetragen, steht Jonas anschließend an Index 3 zwischen Lea und Finn.

Implementieren Sie die Operation nachtragen in Java, ohne die Operation sortieren zu verwenden.

[5 BE]

e) Die Startnummern der Schützen werden aufsteigend sortiert in einer Reihung verwaltet. Zum Auffinden einer Startnummer dient die Klassenoperation **suche** (Abbildung 5).

Dokumentieren Sie im Antwortbogen den Werteverlauf der Variablen lo, hi und mid für die Aufrufe `suche(nummern, 42)` und `suche(nummern, 20)` mit der Belegung aus Abbildung 4. Ein Eintrag erfolgt jeweils an der grau markierten Stelle. Geben Sie jeweils den Rückgabewert an.

Analysieren Sie, wie viele Schleifendurchläufe ein Aufruf von suche bei 100 Startnummern höchstens benötigt, und geben Sie an, wie sich diese Anzahl ändert, wenn sich die Zahl der Startnummern verdoppelt.

[4 BE]

f) Die Operation **besterAusVerein(verein: Zeichenkette): Schuetze** soll den Schützen des angegebenen Vereins mit der höchsten Ringzahl liefern. Haben mehrere Schützen dieses Vereins die höchste Ringzahl, wird derjenige mit dem kleinsten Index geliefert; gibt es keinen Schützen des Vereins, wird null geliefert. Die Operation soll auch für eine unsortierte Reihung korrekt arbeiten. Das Struktogramm in Abbildung 6 enthält drei Fehler.

Analysieren Sie das Struktogramm: Identifizieren Sie die drei Fehler, erläutern Sie jeweils deren Auswirkung und geben Sie ein korrigiertes Struktogramm an.

[4 BE]

Summe: 24 BE

Material

zu allen Aufgabenteilen

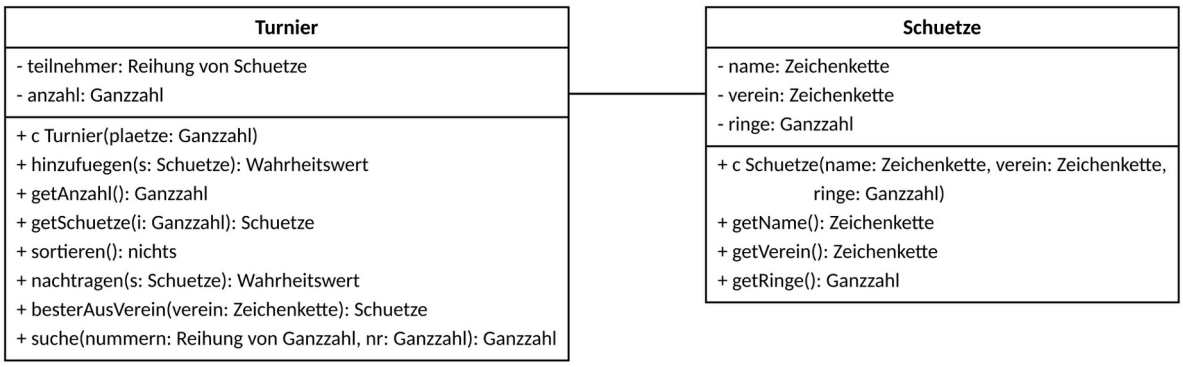


Abbildung 1: Klassendiagramm mit den Klassenkarten der Klassen Turnier und Schuetze

Erläuterungen zu Abbildung 1

- Der Konstruktor Turnier(plaetze) erzeugt die Reihung teilnehmer mit der Länge plaetze und setzt anzahl auf 0.
- Die Schützen stehen lückenlos in den Feldern 0 bis anzahl − 1; alle übrigen Felder enthalten null.
- hinzufuegen(s) legt s im ersten freien Feld ab und liefert wahr. Ist s null oder sind alle Plätze belegt, wird falsch geliefert.
- getSchuetze(i) liefert den Schützen mit dem Index i; bei einem ungültigen Index wird null geliefert.
- sortieren() ordnet die Schützen absteigend nach ihrer Ringzahl (Abbildung 2).
- suche ist eine Klassenoperation: Sie arbeitet nur auf der übergebenen Reihung nummern, nicht auf teilnehmer.

zu den Aufgabenteilen a) bis c)

```
sortieren(): nichts
für i von 1 bis anzahl − 1
    j ← i
    solange j > 0 und teilnehmer[j − 1].getRinge() < teilnehmer[j].getRinge()
        hilf ← teilnehmer[j]
        teilnehmer[j] ← teilnehmer[j − 1]
        teilnehmer[j − 1] ← hilf
        j ← j − 1
```

Abbildung 2: Struktogramm der Operation sortieren der Klasse Turnier

zu den Aufgabenteilen a) bis c) und f)

Index	0	1	2	3	4	5
name	Ida	Ben	Mila	Tom	Lea	Finn
verein	Falkenhorst	Pfeilgilde	Falkenhorst	Eibenhain	Falkenhorst	Pfeilgilde
ringe	47	52	45	58	52	50

Abbildung 3: Belegung der Reihung teilnehmer eines mit new Turnier(8) erzeugten Turniers (anzahl = 6; Felder 6 und 7 enthalten null)

zu Aufgabenteil e)

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Wert	4	9	13	17	21	26	30	35	38	42	47	51	56	60	64

Abbildung 4: Belegung der Reihung nummern (aufsteigend sortiert, Länge 15)

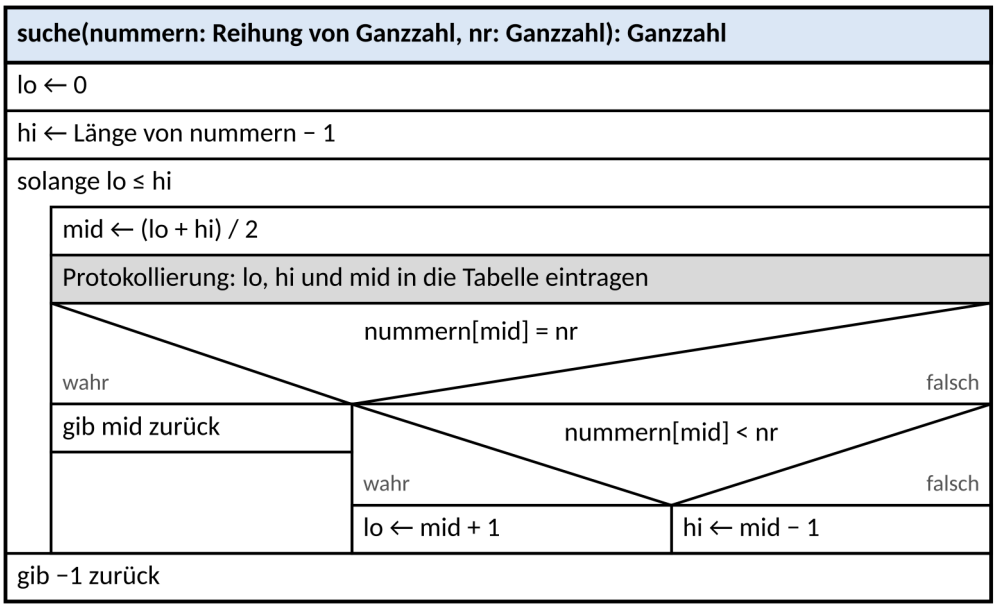


Abbildung 5: Struktogramm der Klassenoperation suche. Hinweis: Die Division von Ganzzahlen ist als Division ohne Rest zu verstehen.

zu Aufgabenteil f)

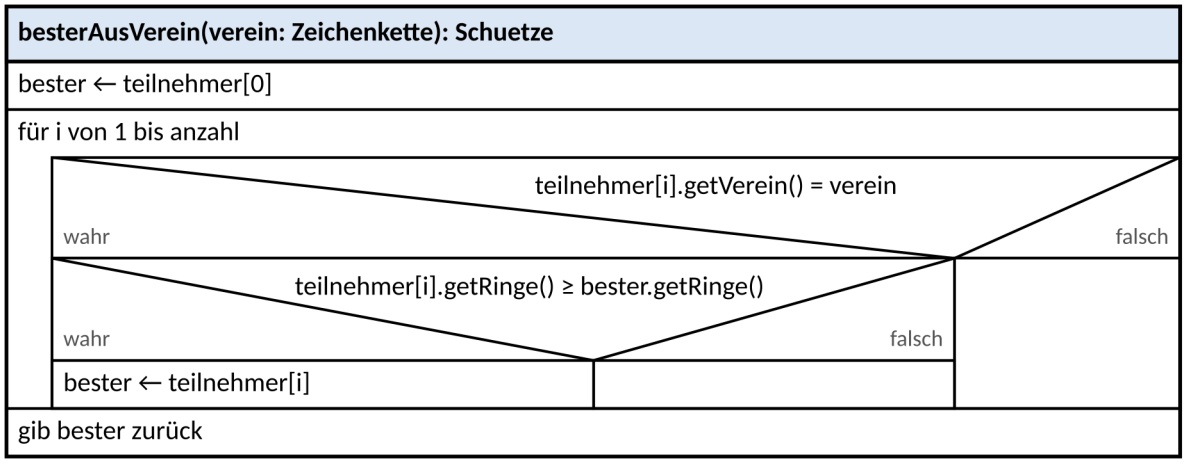


Abbildung 6: fehlerhaftes Struktogramm der Operation besterAusVerein der Klasse Turnier

Antwortbogen

a) Belegung von teilnehmer nach jedem Durchlauf der äußeren Schleife **[4 BE]**

nach Durchlauf	[0]	[1]	[2]	[3]	[4]	[5]
Start	Ida 47	Ben 52	Mila 45	Tom 58	Lea 52	Finn 50
i = 1						
i = 2						
i = 3						
i = 4						
i = 5						

b) Sortierverfahren, Funktionsweise, Begründung zu Ben und Lea **[3 BE]**

c) Vergleiche zählen, ungünstigster Fall, Beurteilung (Operationen und Speicher) **[4 BE]**

Ringzahl-Vergleiche bei Abbildung 3: _____ ungünstigster Fall, 6 Schützen: _____

ungünstigster Fall, n Schützen: _____

Belegung der Ringzahlen im ungünstigsten Fall: _____

d) Implementierung von nachtragen – Papier-Compiler: sauber und lesbar schreiben! **[5 BE]**

e) Werteverlauf von lo, hi, mid · maximale Anzahl der Schleifendurchläufe **[4 BE]**

suche(nummern, 42)		
lo	hi	mid

suche(nummern, 20)		
lo	hi	mid

Rückgabewert suche(nummern, 42): _____ Rückgabewert suche(nummern, 20): _____

f) Fehleranalyse und korrigiertes Struktogramm von besterAusVerein **[4 BE]**

f) Fortsetzung: korrigiertes Struktogramm

Zeichenfläche für das korrigierte Struktogramm (Kopf mit Signatur nicht vergessen)

Auswertung (durch die Lehrkraft oder Selbst-/Peer-Korrektur nach Kernpunkten)

Teilaufgabe	a)	b)	c)	d)	e)	f)	Summe
max. BE	4	3	4	5	4	4	24
erreicht							

Korrigiert von: _____ Papier-Compiler d): ☐ grün beim 1. Versuch ☐ grün nach Korrektur ☐ nicht grün